National Textile University, Faisalabad

Department of Computer Science

Name:	Samar Khalid
Class:	**BSCS-5B**
Registration No:	**23-NTU-CS-1091**
Assignment:	**2**
Course Name:	**Embedded IOT systems**
Submitted To:	**Sir Nasir Mehmood**
Submission Date:	**16-12-2025**

Question-1

ESP32 Webserver (webserver.cpp)

Part A: Short Questions

1.What is the purpose of Webserver server (80); and what does port 80 represent?

The purpose of **Webserver server 80** is to create a local web server on esp-32 that handles http requests from a web browser. Port 80 is default port for http communication, from where users can access the esp-32 web page by entering IP address of ESP-32 on web browser.

- Esp-32 connects to Wi-Fi router to get an IP address.
- Router provides IP address to Esp-32.
- Esp-32 starts its own webserver using web server server 80.
- User open web browser on mobile or laptop.
- User enters IP address in web browser.
- Browser send request to ESP-32.
- ESP-32 responds with an html page.

2. Explain the role of server.on("/", handleRoot()) in this program.

The command **Server.on("/", handleRoot())** tells the ESP-32 web server what to do when homepage is requested. The symbol "/" represents root or home page. So, when any user enters the IP address of ESP-32 in web browser, server calls the function "handleRoot()" that generates and sends the HTML web page with sensor data.

Server.on	It means whenever the request arrives
/	Represents home page or root page.
handleRoot()	Functions that are called when IP address of esp-32 web server is entered in web browser.

3. Why is server.handleClient(); placed inside the loop() function? What will happen if it is removed?

Importance of server.handleClient() placement in loop() function:

The server.handleClient() is placed inside the loop() function as it handles the incoming HTTP requests from web browser. Every time a browser requests webpage, this function checks that request and then calls the appropriate function to handle that request.

Removal:

If we remove server.handleClient() from loop(), ESP-32 web server will no longer respond to http requests from web browser. The webpage will not load and update as the server will no longer respond to incoming HTTP requests.

4. In handleRoot(), explain the statement: server.send(200, "text/html", html);

This statement **server.send(200, "text/html", html);** is used to send the response from ESP-32 web server to web browser (client).

200	This is HTTP status code that indicates that request was successful.
Text/html	This part of command tells the browser that the content being sent was html webpage
html	This is actual content of webpage, including temperature and humidity data.

Conclusion:

This is simple command that sends the HTML webpage to browser so that user can view that sensor data.

5. What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside handleRoot()?

Aspects	Displaying Last Measured Values	Taking Fresh DHT Reading
Data Source	Use previously stored temperature and humidity values.	Reads the sensor in real-time, every time webpage loads.
Speed/Response time	Faster, as no new sensor reading is required.	Slower, as sensor take time in taking new values.
Sensor usage	Minimal	Higher

Webpage load impact	Loads quickly	May take time due to new and fresh reading
Accuracy	May be outdated readings	Real-time readings

Part B: Long Question

Describe the complete working of the ESP32 webserver-based temperature and humidity monitoring system.

The ESP-32 webserver-based temperature and humidity monitoring system is an IOT project that is used to measure the environmental condition using DHT11 sensor, display data locally on OLED and provide the data through web browser.

ESP32 Wi-Fi connection process and IP address assignment:

- First the ESP-32 is connected to Wi-Fi using SSID and password.
- When ESP-32 is connected to Wi-Fi router, it assigns the IP address to ESP-32.
- This IP address allows the ESP-32 to be accessed by any device on the same network through web browser.

Web server initialization and request handling:

- A web server is created on ESP-32 using the command webServer server 80 on port 80. This port 80 is default HTTP port.
- The line **server.on("/", handleRoot);** tells the ESP-32 web server what to do when homepage is requested. The symbol "/" represents root or home page. So, when any user enters the IP address of ESP-32 in web browser, server calls the function "handleRoot()" that generates and sends the HTML web page with sensor data.
- The function **server.handleClient();** inside the loop() function continuously listens the incoming HTTP requests and then call the appropriate function to handle that requests.

Button-based sensor reading and OLED update mechanism:

- A push button is connected to ESP-32 for manual sensor reading.
- When this push button is pressed, ESP-32 starts reading the temperature and humidity using DHT11 sensor.
- These values are displayed on OLED and store in global variable for latre use by web server.

Dynamic HTML webpage generation:

- The handleRoot() function generates an HTML webpage dynamically that includes the current dht11 sensor reading.
- If valid data is available, webpage shows the temperature and humidity values.
- If no data is available, it tells the user to press the button to get valid temperature and humidity values.
- The HTML content is sent to the browser using server.send(200, "text/html", html);.

Purpose of meta refresh in the webpage:

The html page contains meta refresh tag that automatically reloads the webpage every few seconds

This ensures that the user sees the updated sensor readings without manually refreshing the webpage.

Common issues in ESP32 webserver projects and their solutions:

Problems	Solution
ESP-32 cannot connect to Wi-Fi	**Check SSID and password, check Wi-Fi connection**
Webpage doesn't load	**Verify the IP address of ESP-32and also check the server.handleClient() is in loop().**
Sensor reading fails	**Check DHT11 sensor wiring**
Webpage update slowly	**Avoid taking fresh sensor readings on every request; use last measured values when possible**
OLED doesn't display	**Check I2C connections**

Question-2

Blynk Cloud Interfacing (blynk.cpp)

Part-A: Short Questions

1. What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?

- The blynk template ID identifies the specific device template created in blynk Cloud. It defines the structure of the project that include virtual pins,widgets and datatypes.
- The ESP-32 must use the same template ID as cloud template to ensure it matches the expected layout and can communicate correctly with the blynk app.
- If it doesn't match, data may not properly display on the app and server may reject the device.

2. Differentiate between Blynk Template ID and Blynk Auth Token.

Feature	Blynk Template ID	Blynk Auth Token
Purpose	Identifies the device template in blynk Cloud	Identifies specific device instance
Usage	Ensure ESP-32matches the cloud template structure	Use to authenticate the device to blynk server.
Scope	Share by all devices using same template	Unique for every physical device.

3. Why does using DHT22 code with a DHT11 sensor produce incorrect readings? Mention one key difference between the two sensors.

DHT22 and DHT11 both use different data range and protocols and ranges.

If we use DHT22 code with DHT11 sensor it produces incorrect temperature and humidity readings.

Key Difference:

Range	DHT22	DHT11
Temperature	-40–80°C	0–50°C
Humidity	0–100%	20–90%
Precision	High	low

4. What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?

- Virtual pins are software defined pins in blynk used for sending and receiving data between the device and the app.
- They do not correspond to the physical hardware pins on the ESP-32.

Prefer over GPIO pins:

- Multiple devices can share the same virtual pins without conflicts over wiring.
- Safer and flexible for communication over cloud.
- Allows dynamic data transfer like sensor readings, commands, or notifications.

5. What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?

BlynkTimer allows scheduling of non-blocking tasks at regular interval.Delay() pauses the program but BlynkTimer doesn't stop any program.ESP-32 can continue handling other task like Blynk communication, button presses, and other task simultaneously.

Example in code:

timer.setInterval(5000L, periodicSend), sends data every 5 seconds without blocking the loop.

Part B: Long Question

Explain the complete workflow of interfacing ESP32 with Blynk Cloud to display temperature and humidity values.

The ESP32 Blynk system is an IOT project in which ESP-32 reads temperature and humidity from DHT11 sensor, then it displays data on OLED screen and sends the data to Blynk Cloud for monitoring.

Creation of Blynk Template and Datastreams:

- **A Blynk** Template is created in the blynk Cloud , defining the device structure, widgets, and virtual pins.
- Datastreams are created for each type of data (for temperature it is V0 an for humidity its is V1).
- These datastreams define how data will be visualized in the blynk app.

Role of Template ID, Template Name, and Auth Token:

Name	Role
Template ID	**It identifies the device template in the blynk Cloud.Ensure that ESP-32 matches the cloud template for proper communication.**
Template Name	**A human readable name (dht) of template that can be identified easily**
Auth Token	**It is unique for every device. Used to authenticate the ESP32 to Blynk Cloud so it can send and receive data.**

Sensor configuration issues (DHT11 vs DHT22):

DHT22 and DHT11 both use different data range and protocols and ranges.

If we use DHT22 code with DHT11 sensor it produces incorrect temperature and humidity readings.

Key Difference:

Range	DHT22	DHT11
Temperature	-40–80°C	0–50°C

Humidity	0–100%	20–90%
Precision	High	low

Sending data using Blynk.virtualWrite():

Data from DHT sensor is read periodically or manually by pressing button.The values are sent to Blynk cloud using virtual Pins:

Blynk.virtualWrite(V0, temperature);

Blynk.virtualWrite(V1, humidity);

Without the need for physical GPIO pins, virtual pins enable flexible, software-defined communication. The data is then shown in widgets specified in the template on the Blynk application.

Workflow:

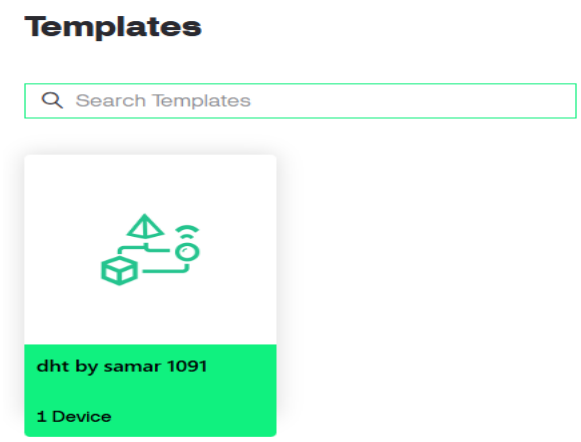
- ESP-32 connects to the Wi-FI using SSID and password.
- ESP-32 authenticates with Blynk Cloud using Auth Token
- ESP-32 reads temperature and humidity from DHT22 sensor.
- Data is displayed on OLED for local monitoring.
- Data is sent to Blynk Cloud using Blynk.virtualWrite() for remote visualization.
- Button press triggers immediate reading and upload.
- Blynk app recieves data and updates widgets in real time.

Common problems faced during configuration and their solutions:

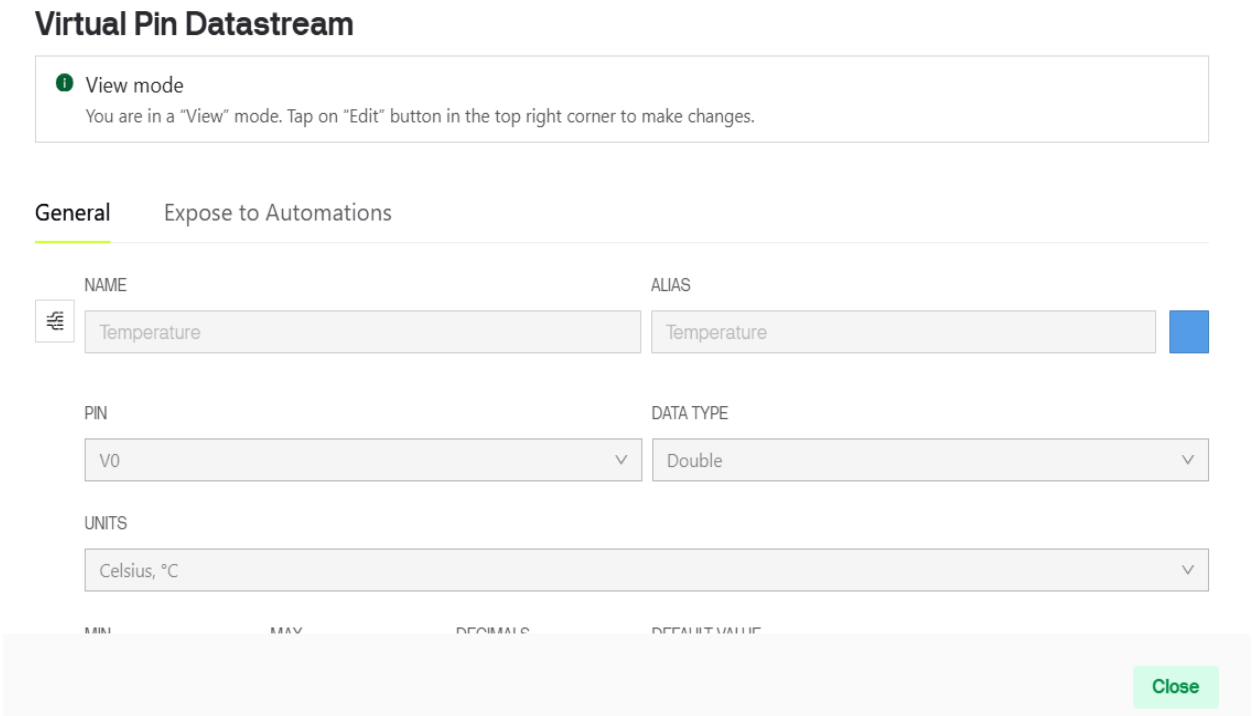
Problem	Solution
ESP-32 cannot connect to Wi-Fi	
Device not appearing in Blynk app	Verify Auth Token matches the device.
Sensor readings fail	Check wiring
Blynk app not updating	Ensure Blynk.run() and timer.run() are called in loop()
Button press not working	Check GPIO pins

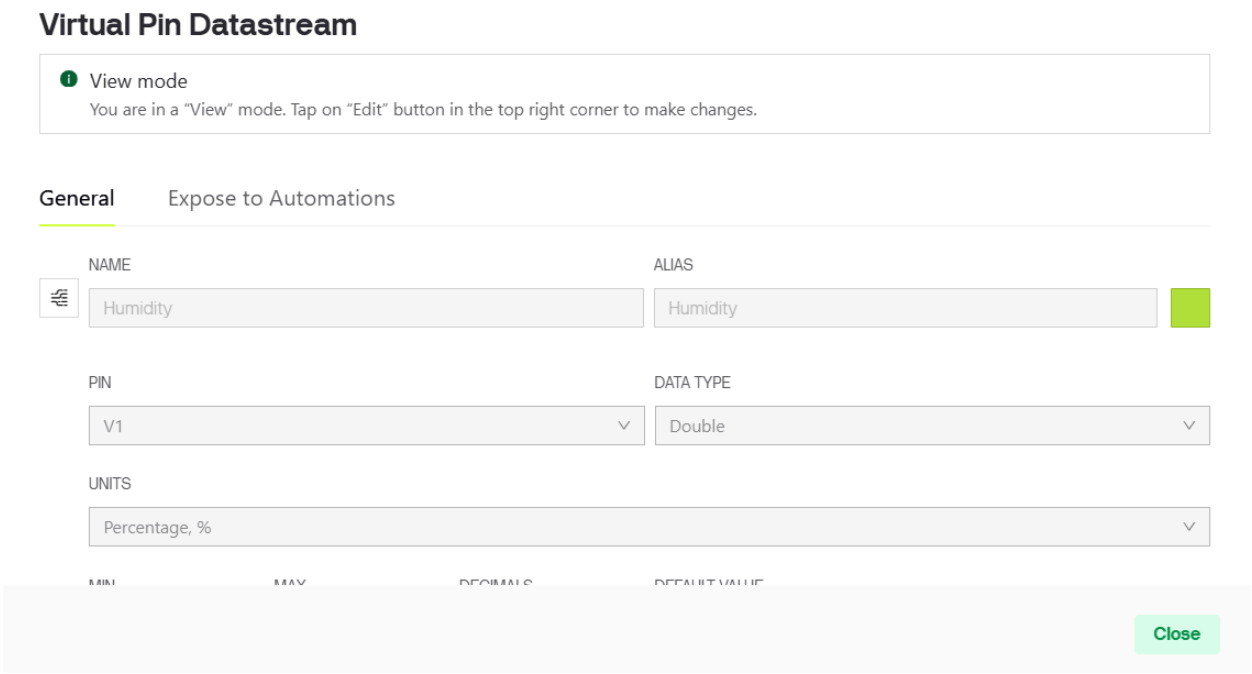
Screenshot:

Template:

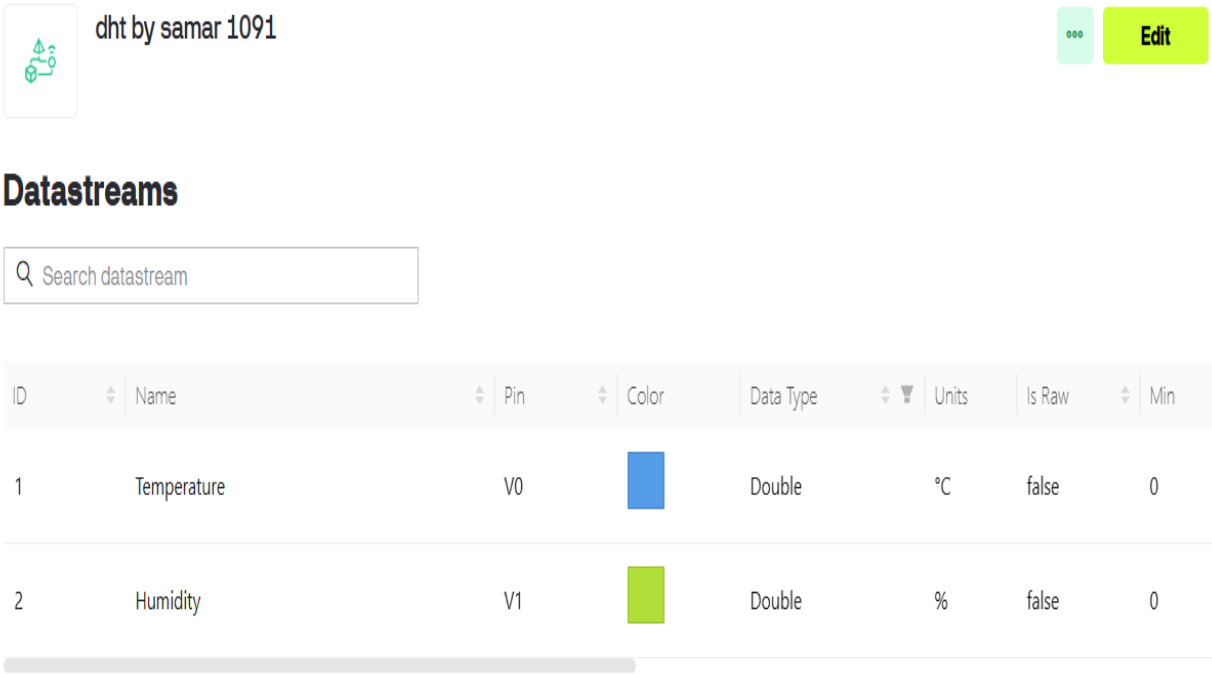


Virtual Pin:

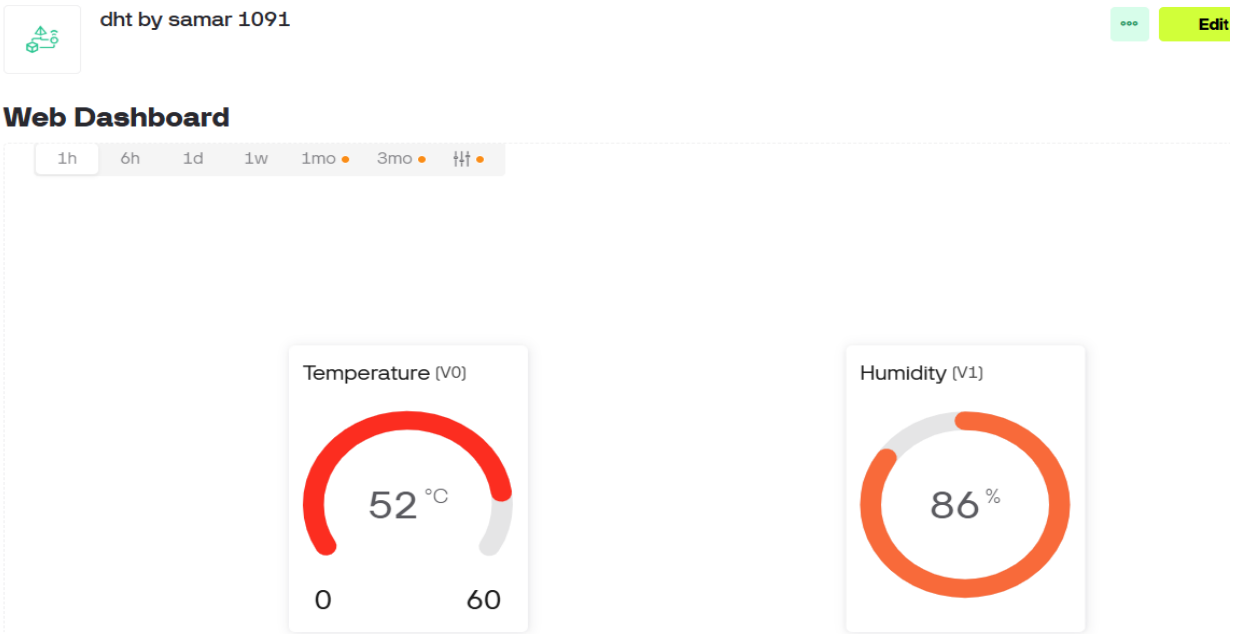




DataStreams:



Web DashBoard:



Mobile App:

Git Repository Link:

https://github.com/samarkhalid1091/embedded_systems_BSCS-5B_1091